Two ideas explained: the last-two predictor and the shadow replay

Part A. The Tsafrir predictor ("average of the user's last two jobs")

Where it comes from. Tsafrir, Etsion and Feitelson, *IEEE Transactions on Parallel and Distributed Systems*, 2007. Supercomputer schedulers need to know how long each job will run, so users must state an **estimate** when they submit a job. Users guess badly: they play safe and over-ask, and they pick round numbers (the paper reports that 90% of jobs use the same 20 values). In our SP2 data the typical job runs for only 18% of the time requested. The paper replaces the user's guess with a **prediction** made by the system from the user's own history.

The rule (3 steps).
1. Guess = the **average runtime of the user's two most recently submitted jobs that have already finished**.
2. If the user has no finished jobs yet, use the user's own estimate (the request).
3. Never guess more than the user's own estimate (in a real system the job is killed at its estimate anyway).

Why the last two? People repeat similar work, so a user's most recent jobs are the best clue about the next one (the paper shows plots of users doing remarkably repetitive work). Averaging two jobs smooths out a one-off oddball while still reacting quickly when the user starts doing something different. One job is noisy; many jobs are stale because they mix old behaviour with new. The paper's own conclusion is that *recency matters more than similarity*, and in its experiments using fewer past jobs worked better (two slightly better than one).

A worked example: one user whose jobs suddenly become much longer (job 4 onward).

Job	True runtime	Last 1 job	Last 2 jobs (average)	Average of all so far	Our old running average
1	10	request / default	request / default	request / default	8 (default)
2	12	10	10	10	9.0
3	11	12	11.0	11.0	10.5
4	60	11	11.5	11.0	10.8
5	58	60	35.5	23.3	35.4
6	61	58	59.0	30.2	46.7

All rules are fooled by the jump at job 4 (nobody can see it coming). After that, "last 2" catches up within two jobs, "last 1" catches up at once but would also chase every one-off outlier, and "average of all" stays stale. Notice that our **old running average** (half its weight on the newest job, a quarter on the one before, and so on) behaves much like last-two. That similarity matters for the results below.

What the paper does beyond the predictor, and why we skip it. In the paper the estimate is also the *kill time*, and predictions can turn out too short, so it "corrects" a prediction upward when a job outlives it. That is needed for *backfilling* (reserving machine slots from predicted end times). Our scheduler is single-CPU and non-preemptive and only uses a guess to *order* waiting jobs, so correction would change nothing. We implemented steps 1–3 exactly (**TsafrirEstimator**), with the window size adjustable.

How does a better guess make scheduling better? SJF simply runs the job with the smallest guess. If the guesses put jobs in the right *order*, short jobs go first and the average wait drops. If a guess is wrong, a long job jumps the queue and blocks short ones. We measure ordering quality with rank correlation (1 = perfect order): On cold-start SP2 slices the guesses score about 0.5 for the running average and 0.62 for our best estimator, and the waiting time follows.

What OUR data says (an honest correction). We first reported that "last-two makes SJF 33% better." Careful testing showed that was mostly something else:

• **The early gain came from using the request for a user's very first job**, not from averaging. Giving the same old running average that fallback captured almost all of it (1,442 vs 1,394 for the full predictor).

• **Once the estimator has history, last-two is not clearly better than the running average** (864 vs 931 ticks, difference not significant), and window sizes 1, 2, 3, 5, 10 all perform alike on SP2. On Paragon (no requests) one job (k=1) is clearly worse.

• **The clear winner is the corrected estimator** (request × the user's learned actual÷requested ratio): 772 ticks, about 17% better than the running average, and a significant gain over last-two.

So: the last-two rule is a sound, cheap baseline from the literature, but in our experiments the real lever was *using the request, corrected by what each user usually does*.

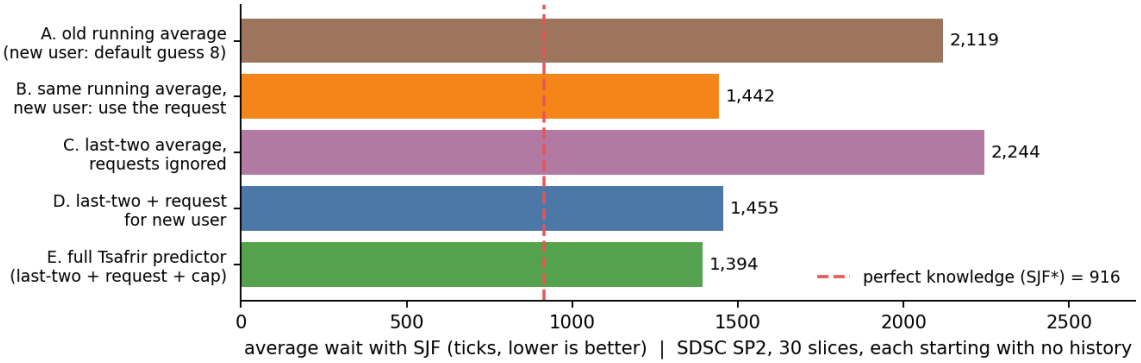


Figure A. Where the early gain came from. Using the request for new users (B) gets almost all of it; averaging two jobs without requests (C) does not help.

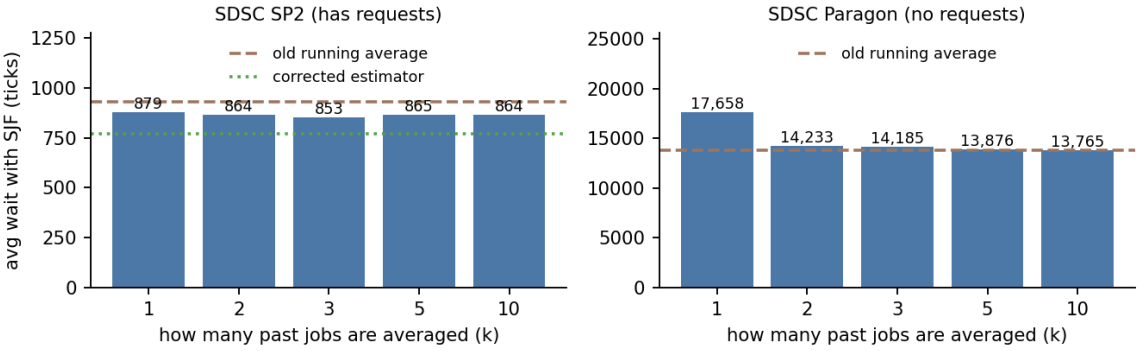


Figure B. With history available, how many past jobs are averaged (k) hardly matters on SP2, and one job is clearly too few on Paragon. Dashed lines: the old running average.

Part B. The "shadow" idea (shadow replay)

In one sentence. Before really changing the scheduling rule, the scheduler runs a **pretend copy (a "shadow") of the recent past** under each rule, and switches only if another rule clearly would have done better. The pretend runs never touch the real jobs, which is why it is called a shadow.

An everyday picture. Before changing the route you drive to work, you check how each route would have done on the last few days' traffic, and you only switch if another route was clearly faster. You do not experiment with today's real commute; you look at a pretend run of the recent past.

The steps, every 10 ticks:

1. Look at the **last 12 jobs that arrived**. Finished jobs keep their real length; unfinished ones use our estimate (all a real scheduler would know).
2. **Re-run those 12 jobs in the background** under each candidate rule: FCFS, SJF and Round Robin (and the rule currently in use).
3. **Compare the average wait** each rule would have given on that window.
4. Switch only if another rule is **at least 5% better, on two checks in a row**. Otherwise keep the current rule.
5. A switch costs 1 tick. The chosen rule then runs the real jobs until the next check.

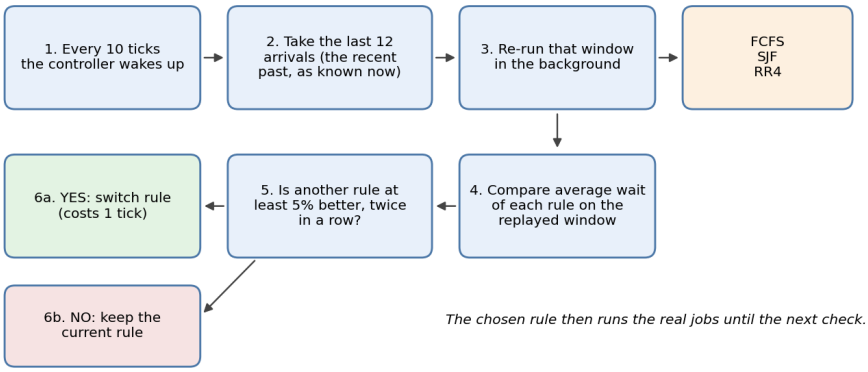


Figure C. One decision cycle of the shadow-replay selector.

What the shadow run sees	Replay scores (average wait)	Decision
One long job plus four short ones (the picture on page 1 of the briefing)	FCFS 10.4, SJF 10.2, **Round Robin 5.8**	Round Robin is far better than 5%: switch, after two checks in a row.
Five long, similar jobs	**FCFS 20.2**, SJF 19.4, Round Robin 36.4	SJF is only 4% better (below the 5% margin): stay on the current rule.

Why the 5% margin and “twice in a row”? They stop it switching on noise. On our tuning seeds, switching after a single sighting (patience 1) scored 24.3 versus 14.1 for two in a row (lower is better).

Strengths and weaknesses (with our numbers).

- **Safety net:** on synthetic workloads it is never the worst, and its worst-case regret is 30% against 60% for plain SJF. It does not need to be told what kind of workload it faces.
- **It is only as good as its guesses:** the replay uses the estimates, and on heavy-tailed real jobs a 12-job window is a noisy sample of the future.
- **With good guesses, plain SJF beats it:** on SP2 with history, SJF scored 765 against 954 for the shadow selector (with the corrected estimator), a significant gap, and similar for the other estimators.
- **Cost:** about one rule change per run, plus the time for the replays (about 12 ms per run in our Python simulator, so a relative cost, not a kernel measurement).

Where it comes from. This is a lightweight form of *portfolio scheduling* (and, because it simulates the candidates, *simulation-driven portfolio scheduling*) from the literature, so we describe it as a careful small-scale evaluation, not a new algorithm.

30-second script for your teammate. “Different jobs need different scheduling rules. Instead of guessing which one, our scheduler replays the last dozen jobs under each rule in the background, like checking yesterday’s traffic on three routes, and switches only when another rule clearly wins twice in a row. It protects us from bad choices, but it only works as well as our guesses of job length, and our best result came from improving those guesses, not from switching.”